

Advanced Natural Language Processing: AdvNLP

*Lesson 6: Training Transformer
Decoders & LLMs for text generation*

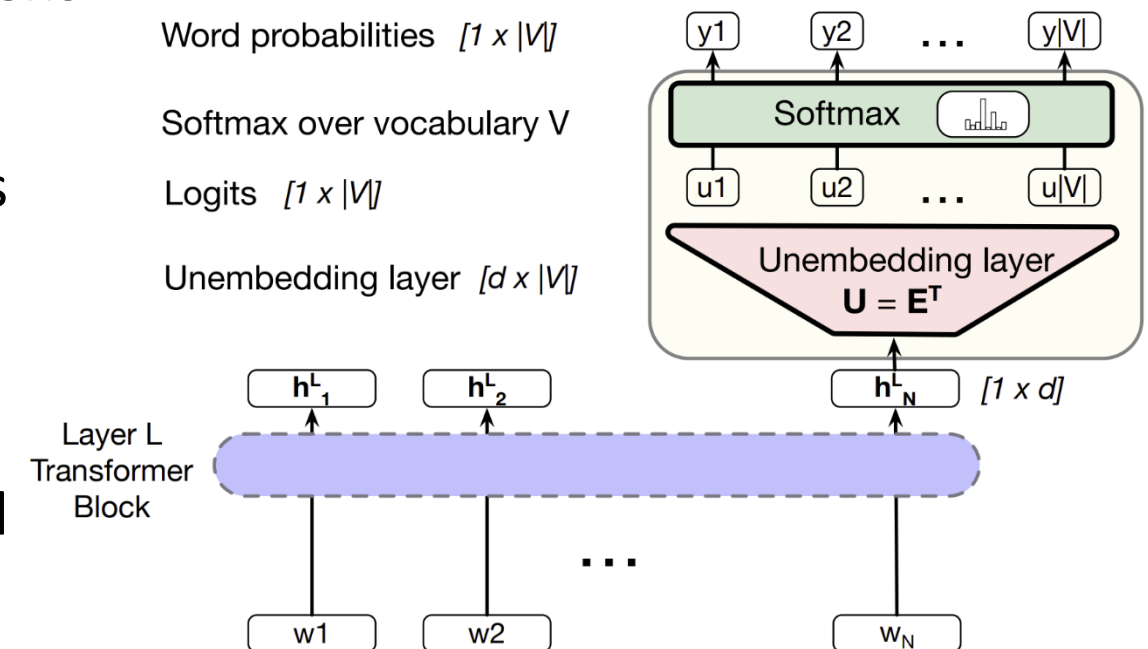
Table of Contents

1. LLM decoding methods
2. LLM instruction-tuning methods
3. LLM fine-tuning methods

1. LLM DECODING METHODS

Text generation with Transformer decoder LMs

- Decoder receives previously generated tokens
- Final transformer block produces a hidden vector mapped linearly to vocabulary logits
- Softmax converts unnormalized logits per token into a probability distribution
- Next token is chosen based on the selected decoding strategy
- Iterative generation is continued until stopping criterion, i.e., EOS token or maximum length is reached

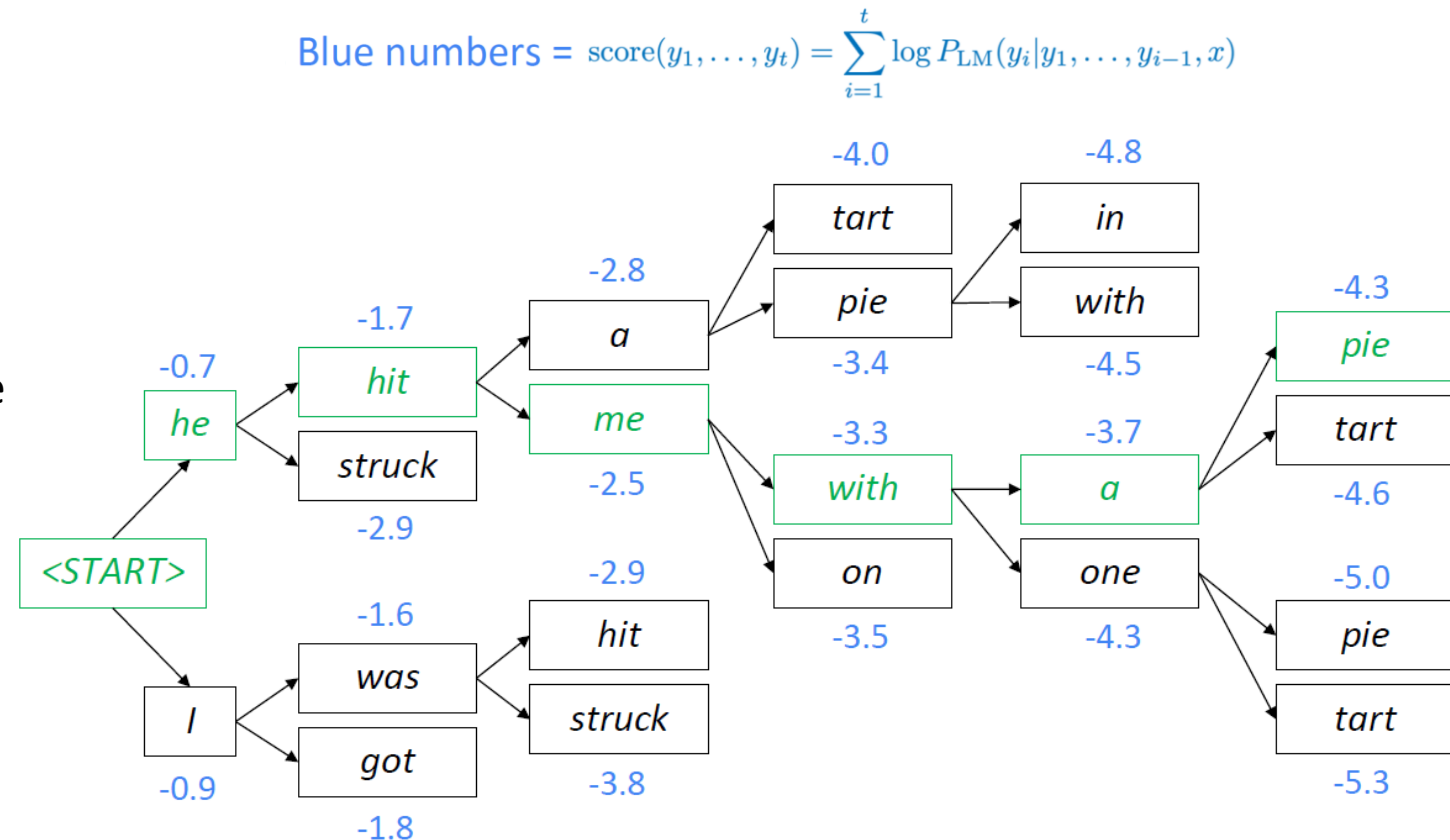


Generating text: “decoding”

- **Decoder models output a probability distribution** over the vocabulary for next token prediction in a sequence. **Decoding** refers to next token selection and **text generation**
- The selection of an appropriate decoding strategy depends on the use case:
 - **Directed generation**: text generation defined through (input, output) pairs, i.e., the output is a constrained transformation of the input (e.g. machine translation, text summarization)
 - **Open-ended generation**: conditional text generation by next word prediction (e.g., story generation, text continuation)
- **Exhaustive search**: explore all possible sequences, examine all tokens... which is computationally intractable (complexity of $\mathcal{O}(|V|^T)$)
- **Greedy search**: at each step, pick the most probable token
- **Beam search**: at each step, pick the most likely k hypotheses and choose the token sequence with the highest probability

Beam search decoding (1)

- Beam size k (here $k = 2$)
- At every step, among the k^2 hypotheses, keep the k ones with highest scores and remove rest
- For each hypothesis, find the top k next words and calculate their scores/logits
- And so on...
- The final top-scoring word and the path leading to it give the solution
- Note, that at inference the model is run k times at each step

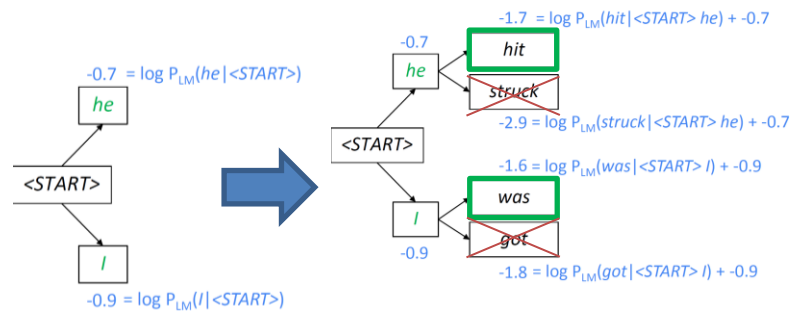


Beam search decoding (2)

Source: «*il m'a entarté*»

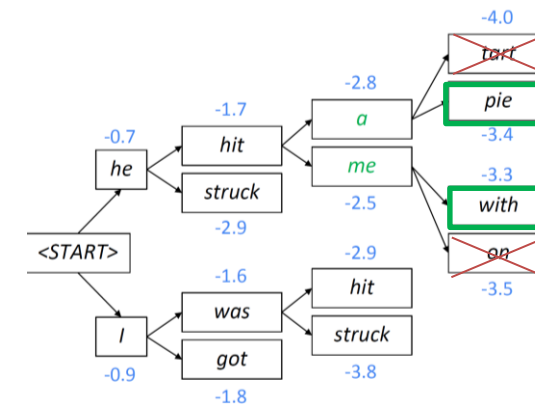
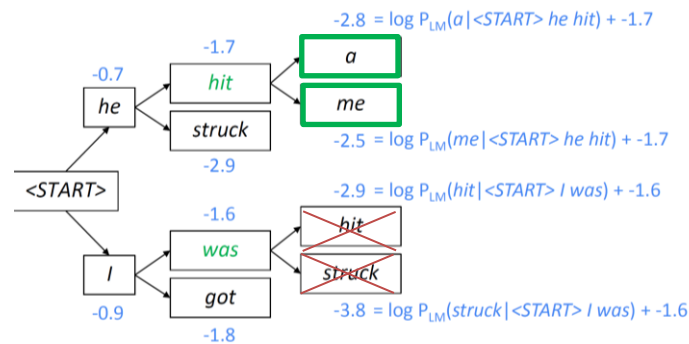


Target: «*he hit me with a pie*»



For each of the k hypotheses, find top k next words and calculate scores

Of these k^2 hypotheses, just keep k with highest scores



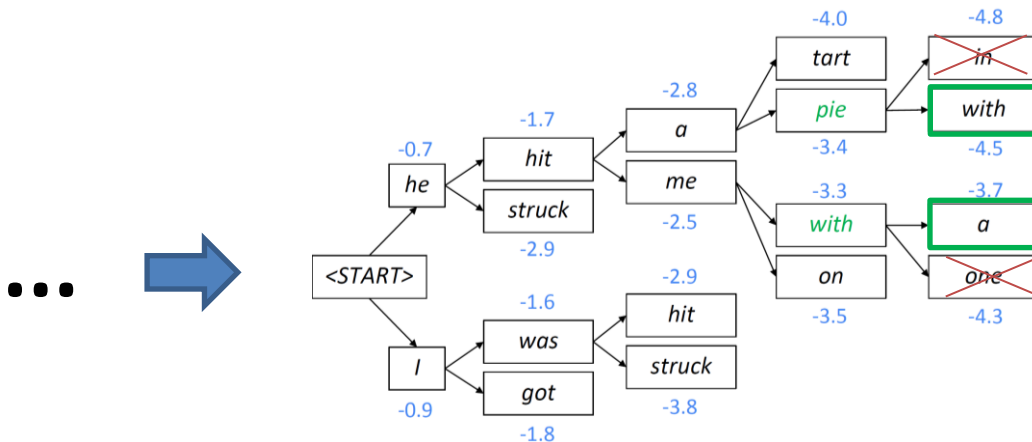
...

Beam search decoding (3)

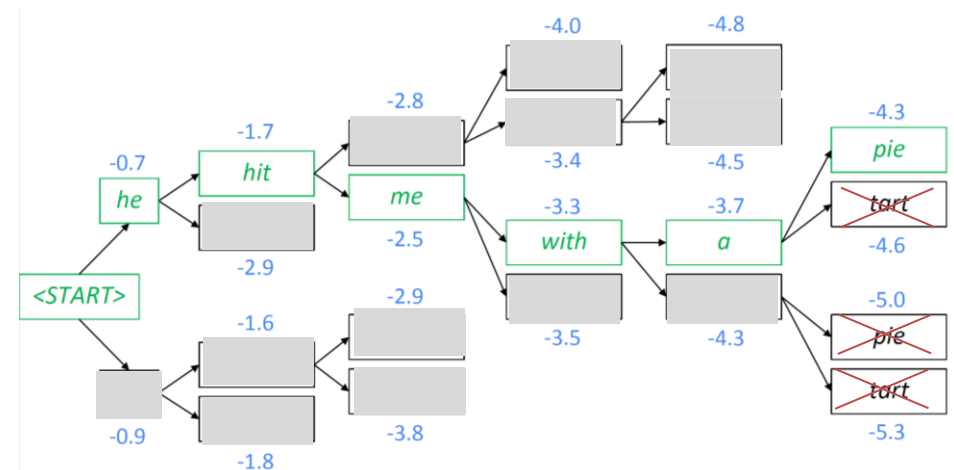
Source: «*il m'a entarté*»



Target: «*he hit me with a pie*»



Backtrack to obtain the full hypothesis



Using beam search decoding means running a model multiple times

Generating text: “decoding”

- **Pure sampling:** at each step, randomly pick the next token according to its predicted conditional probabilities
- **Temperature sampling:** at each step, randomly pick the next token according to probabilities from logits u transformed by temperature t
- **Top-k sampling:** at each step, filter the top-k most likely tokens and pick next token according to re-scaled predicted probabilities
- **Top-p sampling:** at each step, filter the smallest token set with cumulative probability not larger than p and pick next token according to re-scaled predicted probabilities (aka **nucleus sampling**)

Generating text: “decoding”

- Temperature sampling:

- $P(x = V_l | x_{1:i-1}) = \frac{\exp(u_l/t)}{\sum_{l'} \exp(u_{l'}/t)}$
- Sampling with and without temperature t tends to lead to **incoherence**

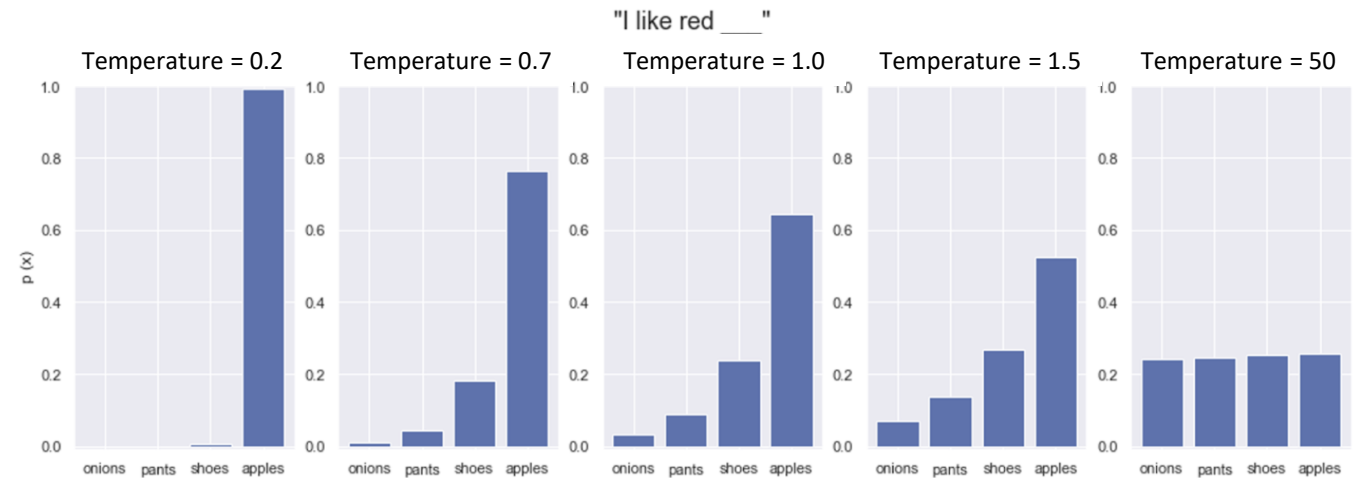


Image from <https://www.hopsworks.ai/dictionary/llm-temperature>

- Top-k sampling:

- small k problematic for flat / large k problematic for peaked distributions. Temperature sampling with $t < 1$ as partial remedy top-k sampling but still problematic w.r.t. **repetitions**

- Top-p sampling:

- size of sampling set adjusted dynamically based on the shape of the probability distribution at each time step, largely resolving Incoherence and repetition

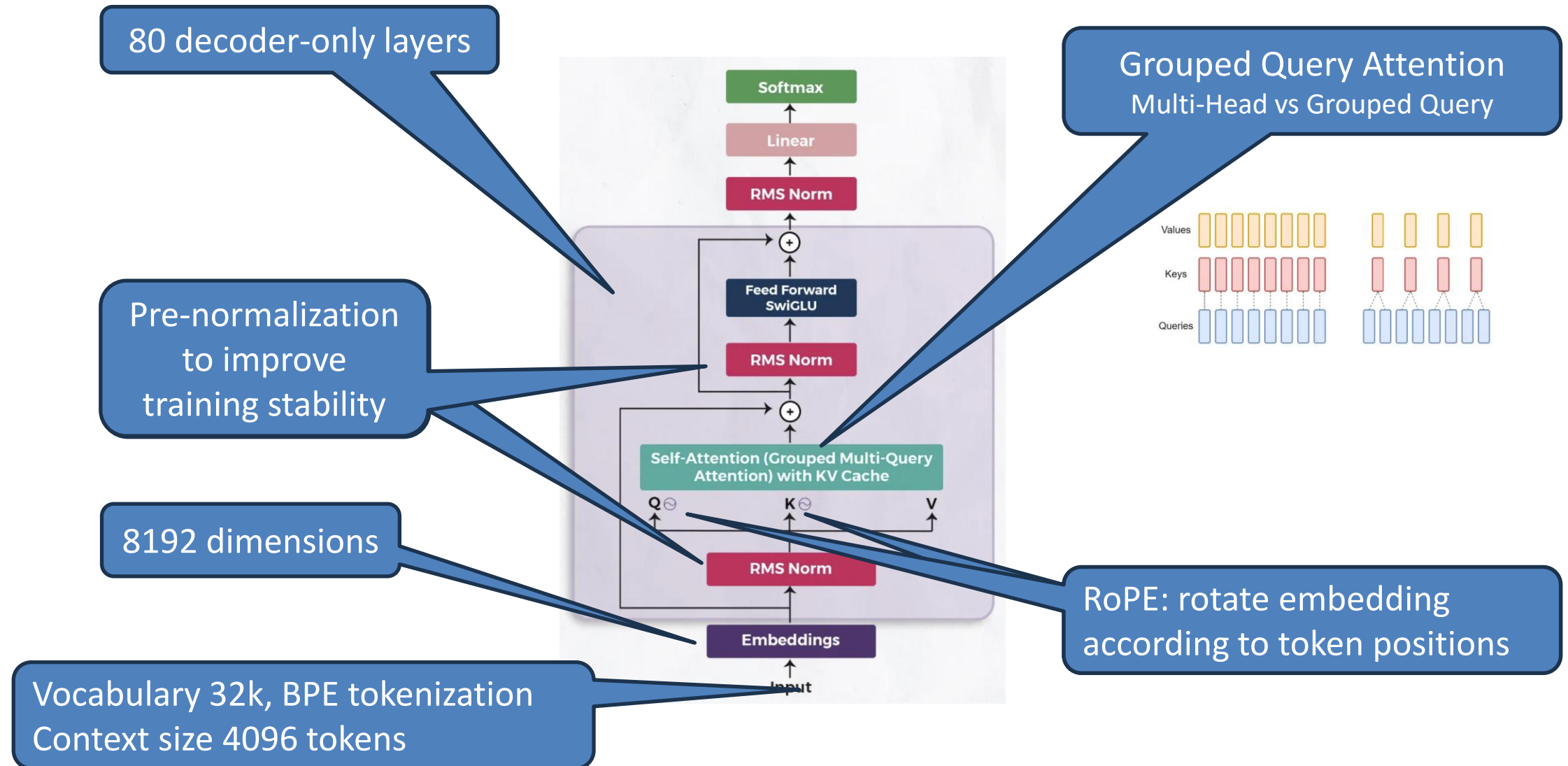
2. LLM INSTRUCTION-TUNING METHODS

Auto-regressive language models

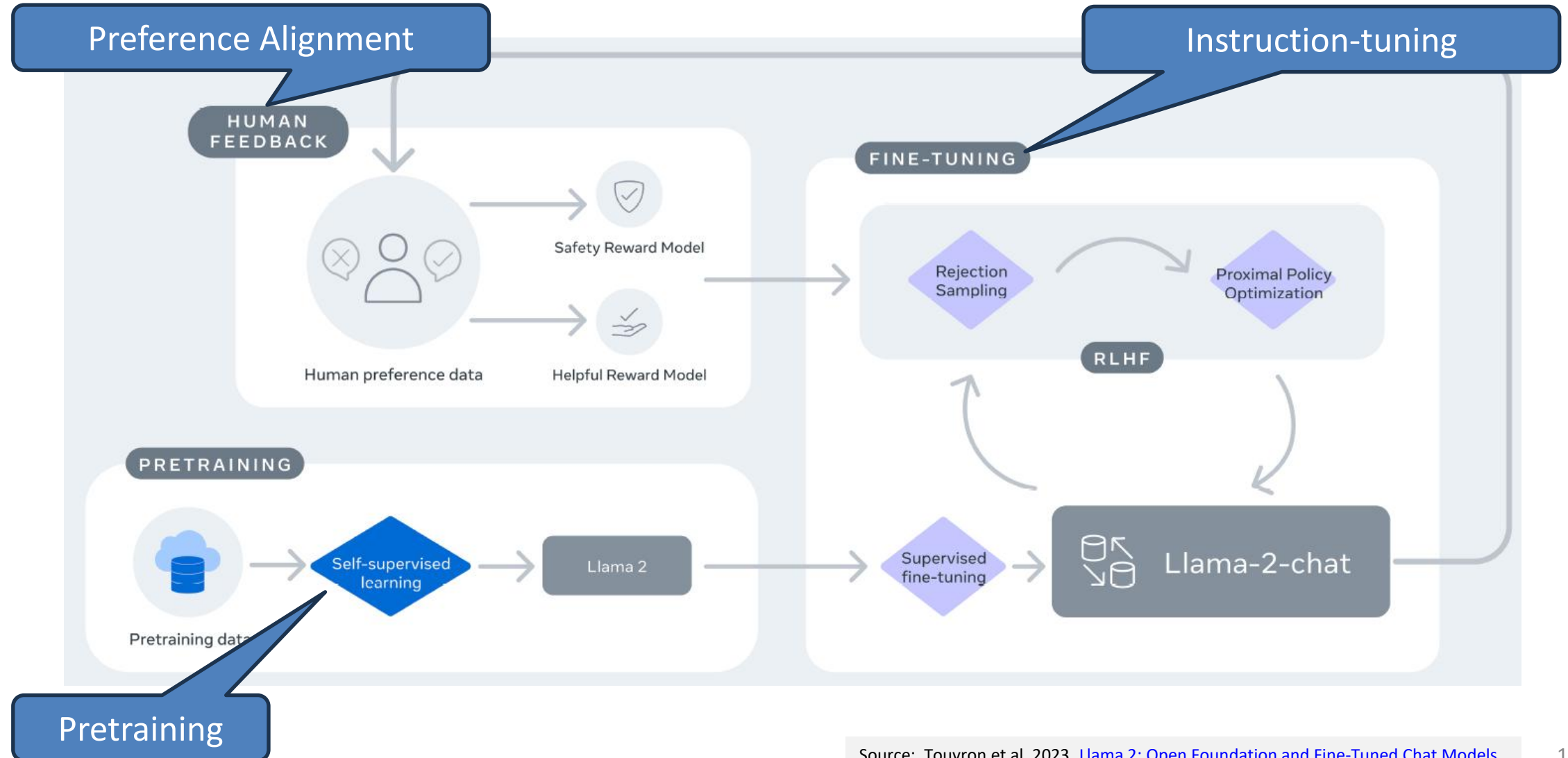
- Large autoregressive LMs include world knowledge
 - learned from their huge training sets & stored in the network's weights
 - Example: complete the following sequence:

The canton of Vaud officially joined the Swiss Confederation in _____
 - However, how can they learn that a **question** must be followed by an **answer**?
 - How can they learn to reply to **instructions other than questions**?
 - How can they learn to provide **safe**, yet **helpful** answers?
- ❖ Next word prediction in Web texts is insufficient for that!

Llama-2 model architecture



Llama-2 training setup



Recap – Pretraining

- Compare predicted next word with ground truth using **cross-entropy as loss function** and **backpropagate loss** with gradient descent
- Always give model the correct history sequence to predict the next word rather than feeding the model its best guess from the previous time step (aka **teacher forcing**)
- Minimization of Cross-Entropy Loss for one token in the sequence

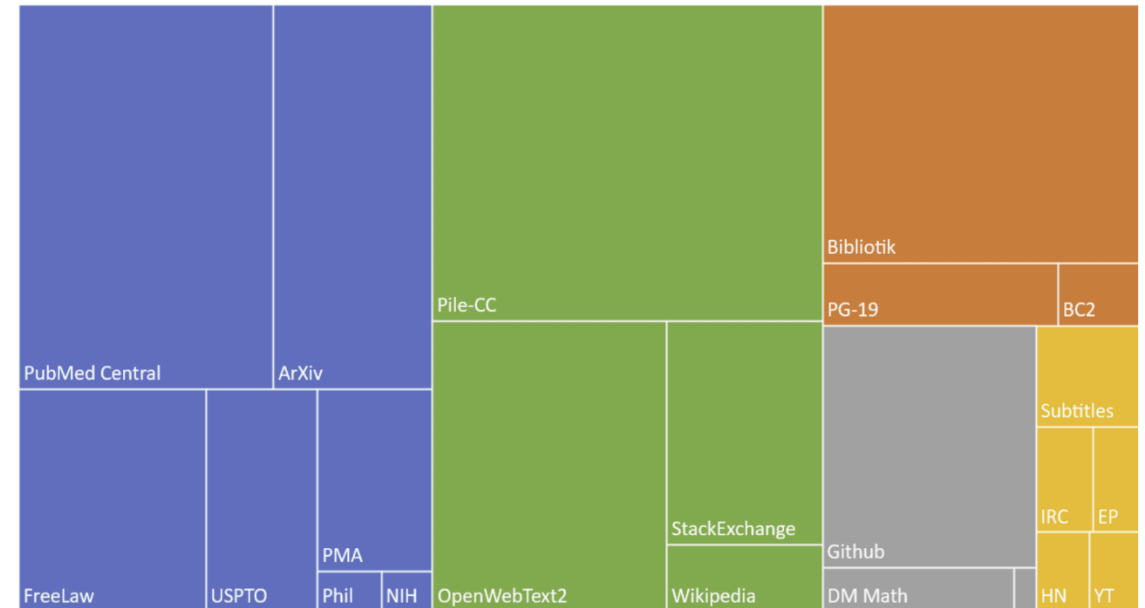
$$L_{CE}(y_t, \hat{y}_t) = - \sum_{w \in V} y_t[w] \cdot \log(\hat{y}_t[w]) = -\log(\hat{y}_t[w])$$

difference between predicted $\hat{y}$ and ground truth y probability distribution for all tokens

due to one-hot encoding since the correct distribution equals 1 for the actual next word only

Recap – Pretraining data

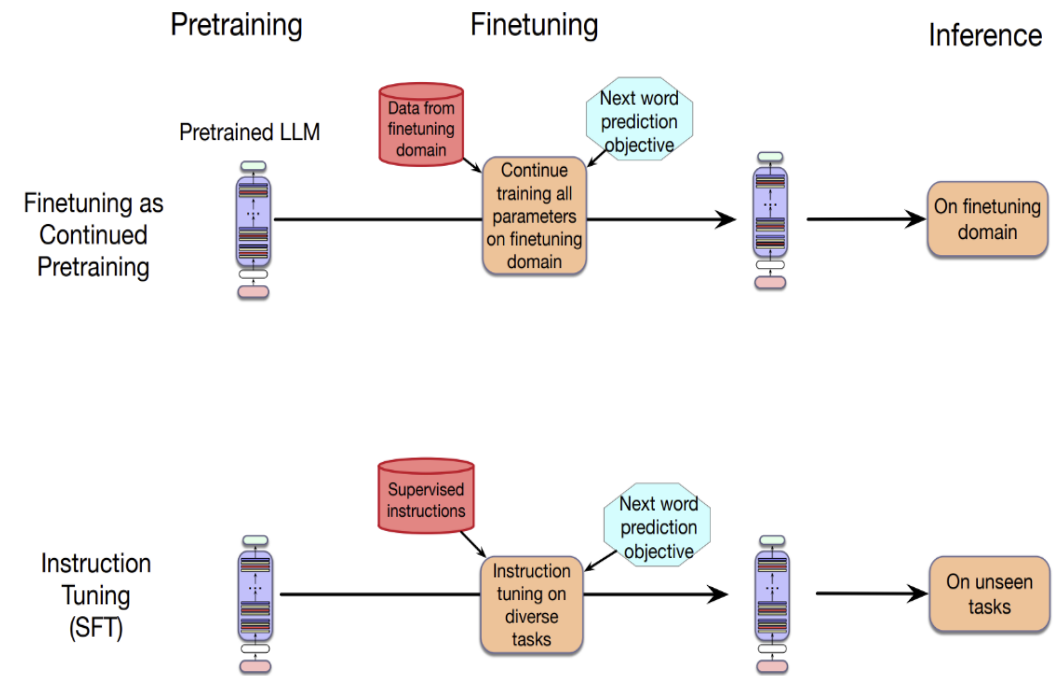
- LLMs are mainly pretrained on text scraped from the web
- Pretraining data is filtered for both quality and safety (e.g., PII, toxicity)
- Using such datasets poses ethical and legal questions (e.g., copyright, consent, privacy, skew)



Pile corpus showing size of different components, color coded by type: **academic**, **incl. patents**, **internet**, **prose incl. books**, **dialogue incl. movie subtitles and chat data**, and **misc**

Instruction-tuning

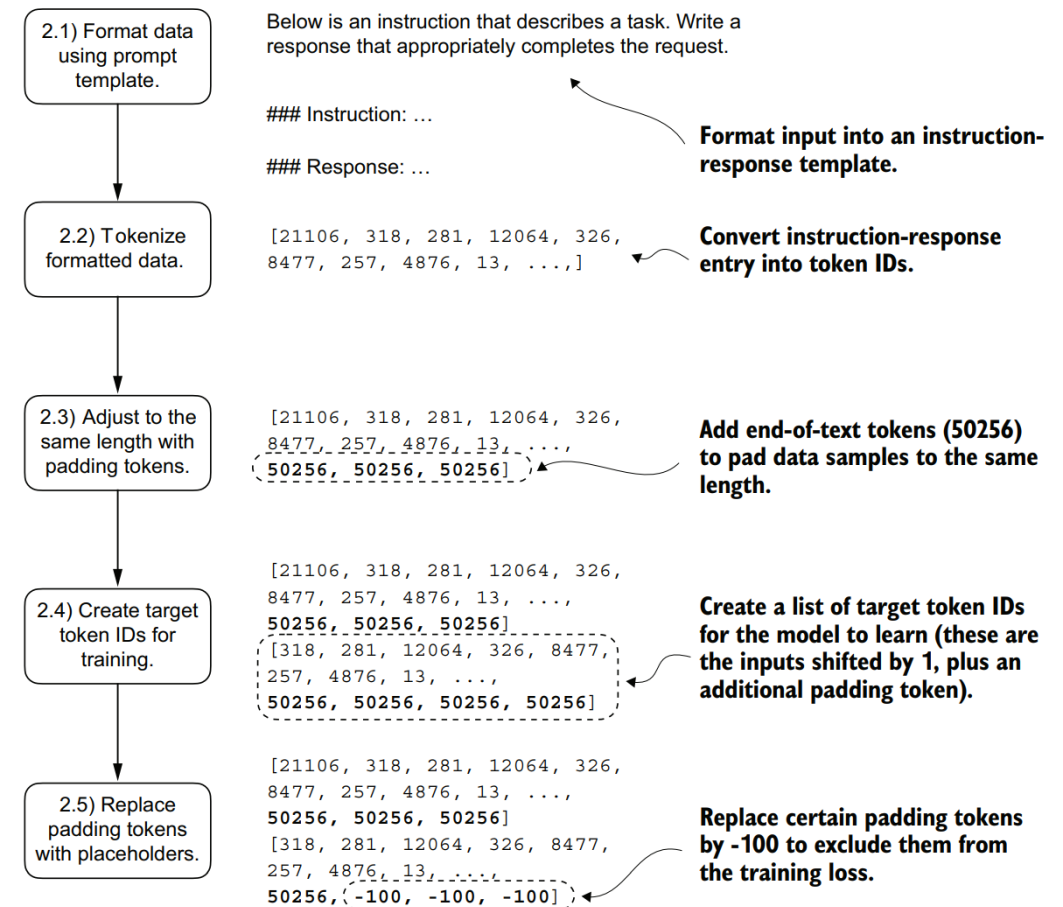
- Instruction-tuning
 - uses a pretrained base LLM and **trains it further by updating all model parameters** on a dataset of instructions plus supervised responses
 - utilizes datasets with **hundreds of 1000s' of instruction instances** with much lower costs than original base LLM pretraining
 - uses the same language model training objective as pretraining, i.e., **next-token cross-entropy**
 - generalize to **follow unseen instructions**, not just memorize tasks seen in the dataset
 - Is **trained on multiple tasks** and **evaluated on withheld task** to ensure generalizability



Instruction is considered supervised fine-tuning even though the objective is self-supervised because each instruction has a correct response

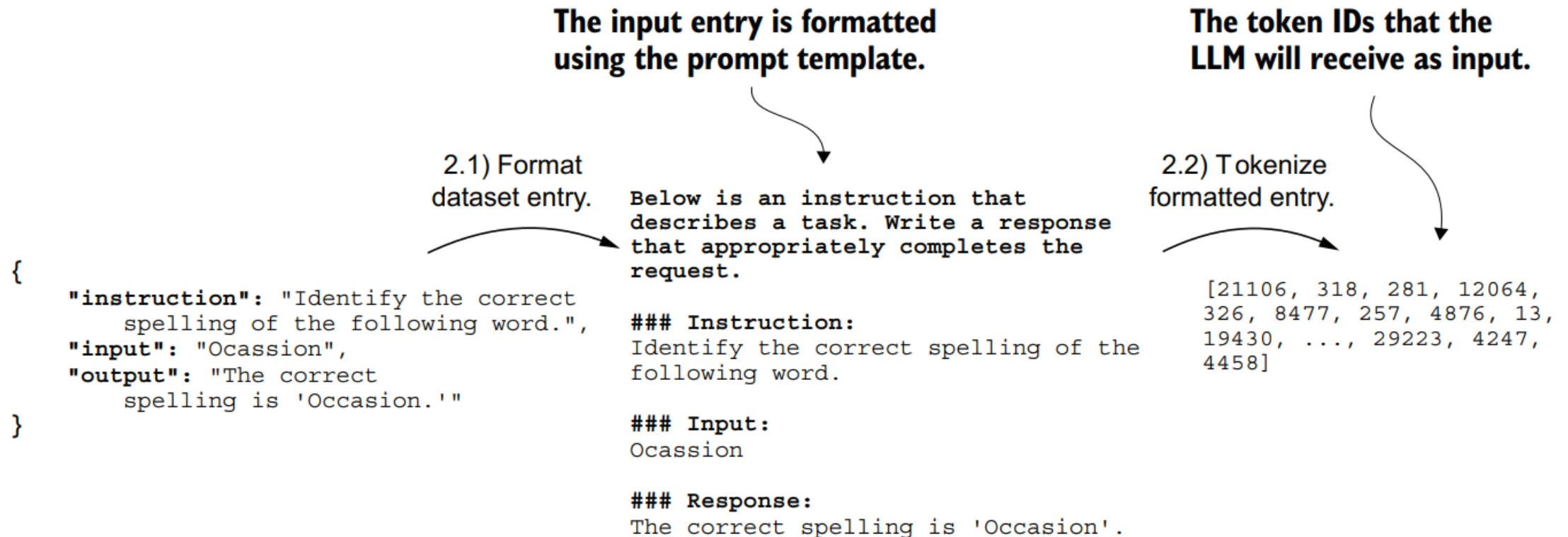
Instruction-tuning

- Instruction-tuning involves 5 steps
 - applying a **prompt template**
 - using **tokenization**
 - adding **padding tokens**
 - creating **target token IDs**
 - Replacing padding tokens by “-100” placeholder tokens to **mask padding tokens** in the loss function



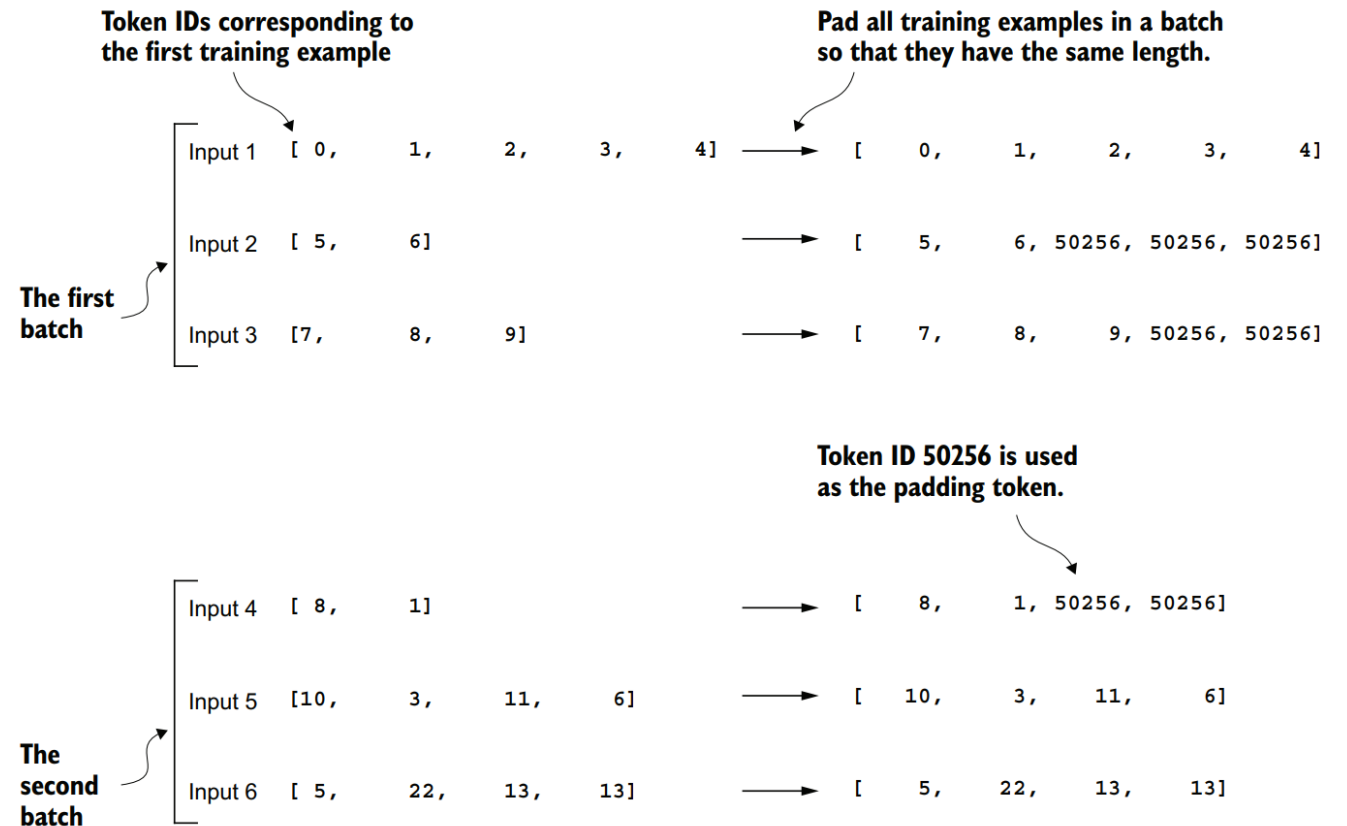
Instruction-tuning

- Instruction-tuning uses a **specific prompt template**



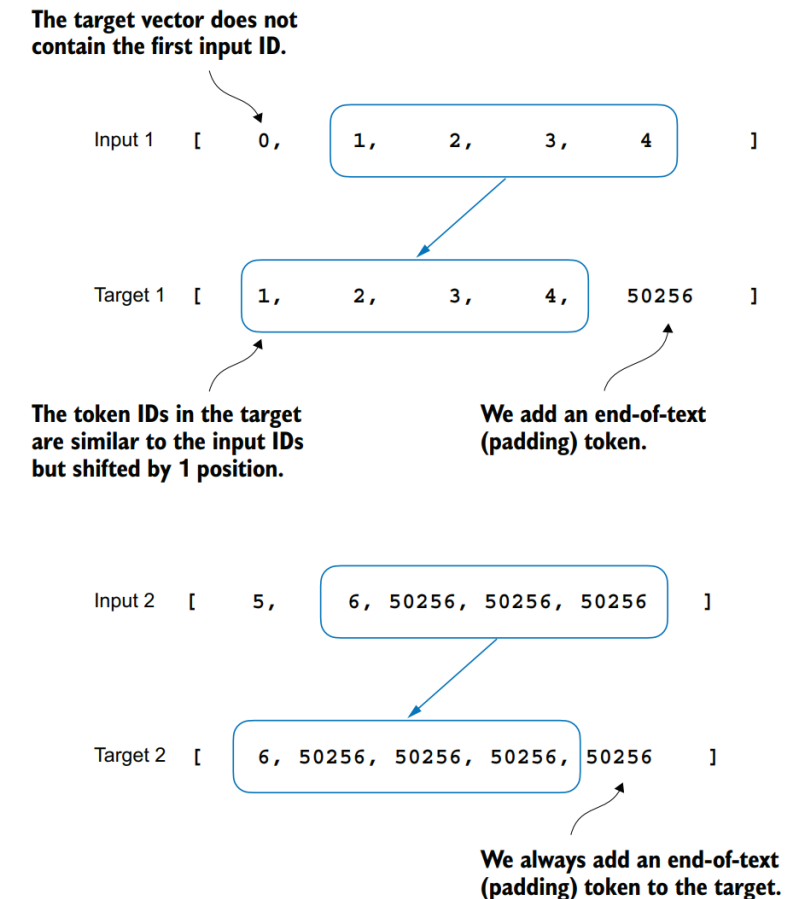
Instruction-tuning

- Instruction-tuning training can be accelerated by collecting multiple training examples in a batch and **padding all inputs to an equal length** with a special padding token (here 50256)
- Note that **sequences per batch typically differ in length**

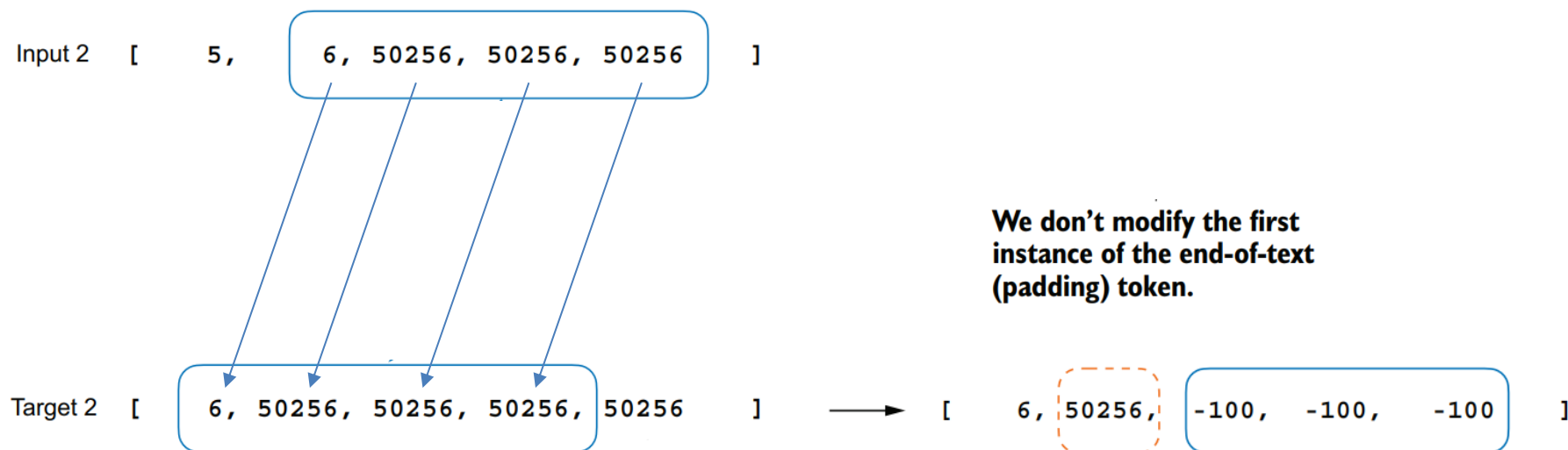


Instruction-tuning

- Similar to LLM pretraining, the **target token** IDs match the input token IDs but are **shifted one position to the right**, so as to predict the next token in a sequence



Instruction-tuning



We don't modify the first instance of the end-of-text (padding) token.

The token IDs in the target are similar to the input IDs but shifted by 1 position.

We always add an end-of-text (padding) token to the target.

We replace all but the first instance of the end-of-text (padding) token with -100.

The target vector does not contain the first input ID.

Instruction-tuning

- During training, the formatted input text is tokenized and fed to the LLM.
- Optionally the **instruction section is masked out**, i.e., token IDs are replaced with the -100 ignore_index value.

Input text:

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
Rewrite the following sentence using passive voice.

Input:
The team achieved great results.

Response:
Great results were achieved by the team.

↓ Tokenize
[21106, 318, 281, 12064, 326, ..., 13]

↖
The token IDs corresponding to the input text

Target text:

is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
Rewrite the following sentence using passive voice.

Input:
The team achieved great results.

Response:
Great results were achieved by the team.<|endoftext|>

↓ Tokenize
[-100, -100, -100, -100, -100, ..., 13, 50256]

↖
The instruction tokens are replaced by -100.

Mask out the instruction when calculating the loss.

Instruction-tuning data

- Instruction-tuning data sets
 - contain a natural language **description** combined with a labeled task **demonstration**
 - cover **many** languages and **tasks** (e.g. sentiment, extractive Q&A, Natural Language Inference)
 - are either **human authored**, extracted from **existing NLP bench tasks with templates**, or **autogenerated using LLMs** and leveraging carefully written **annotation guidelines**

Supervised training data example for NLI

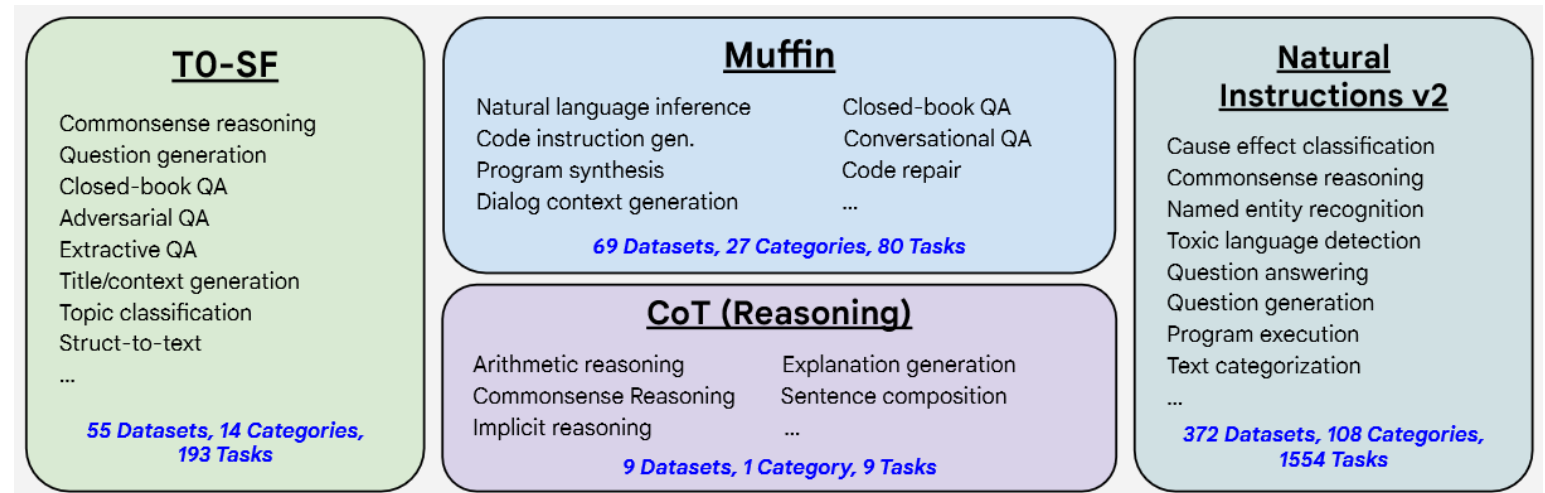
premise	No weapons of mass destruction found in Iraq yet.
hypothesis	Weapons of mass destruction found in Iraq.
label	2

Three different instruction templates for NLI

- 1 -Suppose {{premise}} Can we infer that {{hypothesis}}? Yes, no, or maybe?
- 2 -{{premise}} Based on the previous passage, is it true that {{hypothesis}}? Yes, no, or maybe?
- 3 -Given {{premise}} Should we assume that {{hypothesis}} is true? Yes,no, or maybe?

Instruction-tuning data

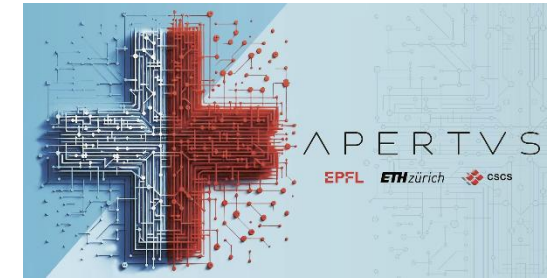
- Google's Flan-PaLM
 - instructions collected from public sources
 - millions of pairs



Source: Chung et al. 2022, [Scaling Instruction-Finetuned Language Models](#)

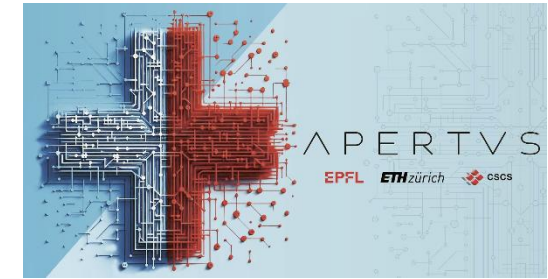
- Meta's Llama-2 (Touvron et al. 2023)
 - based on the observation that many public sets are of insufficient quality
 - collected 27,500 high-quality examples and demonstrate improvement of LM

Instruction-tuning data – Best practice



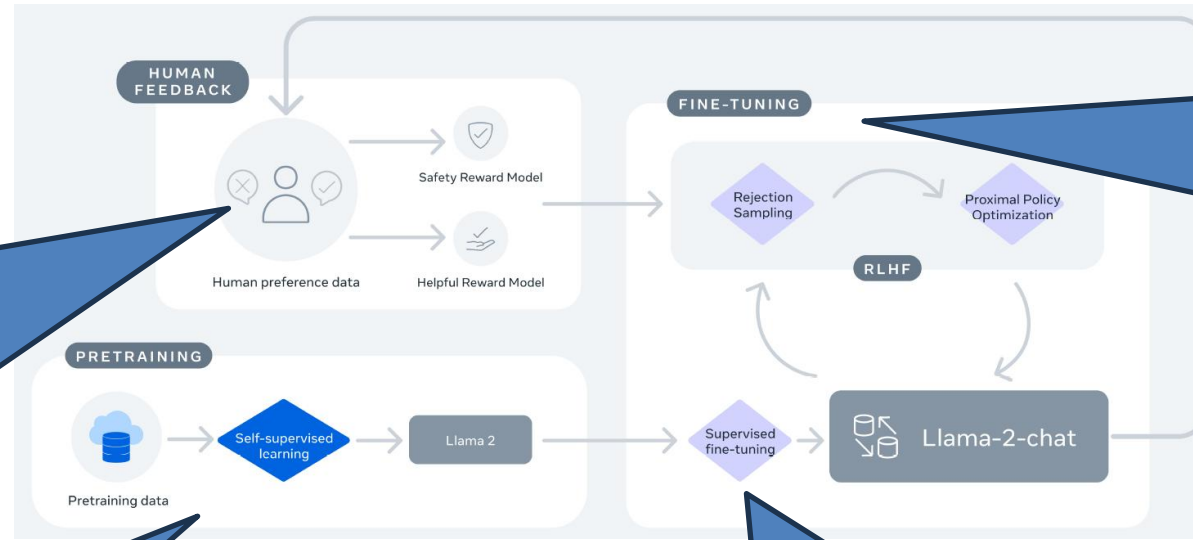
- Use public data exclusively *also used in pre-training*
 - content explicitly released under licenses permitting redistribution and commercial use (e.g., CC-BY, Apache 2.0), and (ii) the collection procedure fully documented and reproducible
 - careful verification required for compound datasets
- Remove PII *also used in pre-training*
 - detection of email addresses, IP addresses, and IBAN bank account numbers via regular expressions search and replacement them with anonymous markers
- Filter toxic & harmful data *also used in pre-training*
 - Classification with trained language-specific models balancing classes via non-toxicity subsampling and covering “Race & origin-based bias”, “Gender & sexuality-based bias”, “Religious bias”, “Ability bias”, and “Violence and abuse”

Instruction-tuning data – Best practice



- Respect AI opt-out (based on robots.txt) *also used in pre-training*
 - Python-based filtering framework that excludes samples or subsets with incompatible licenses using dataset-specific rules
 - Opt-out also in hindsight, not only at crawl time
- Apply quality filtering *also used in pre-training*
 - metadata analysis and manual inspection, including provider provenance and content inspection (e.g., overly long responses, repetitive patterns)
- Apply decontamination
 - Token n-gram matching to identify and remove training samples that are identical or similar to benchmark prompt using overlap

How to train a LM to execute instructions?



3 Reward model learns to predict if a response is *helpful* or *safe* on datasets with binary human ratings ($3 \cdot 10^6$ comparisons, 50% on public data)

4 Fine-tune s.t. model improves its helpfulness and safety scores (using 4 rounds of RS and 1 round of RS+PPO)

1 Pre-training, i.e., learn to predict the next word on a dataset of about 2×10^{12} tokens

2 Instruction-tuning using $\approx 100k$ *instruction-response* data (incl. 28k diverse, high-quality data written by humans)

Principle of RS : sample K outputs from the model and select the best candidate based on reward as gold standard

Some well-known LLMs

(see e.g. <https://lmarena.ai/?leaderboard>
and <https://lifearchitect.ai/models-table/>)

* = open-weights model
** = open-source (training data is known, not necessarily with open license such as CC)

Model(s)	Company
BLOOM**	BigScience Consortium
Claude Sonnet, Opus*	Anthropic
Command-R*	Cohere
DeepSeek*	DeepSeek AI
Dolly*, DBRX*	Databricks
Falcon*	TII UAE
Gemma*, Gemini, Palm	Google
GPT	OpenAI
APERTVS**	ETHZ / EPFL

Model(s)	Company
Granite*	IBM
Grok*	X AI
Llama*	Meta
Mistral*, Mixtral*	Mistral AI
OLMo**, Tülu*	Allen Institue
Phi*	Microsoft
PPLX	Perplexity
Qwen*	Alibaba Cloud

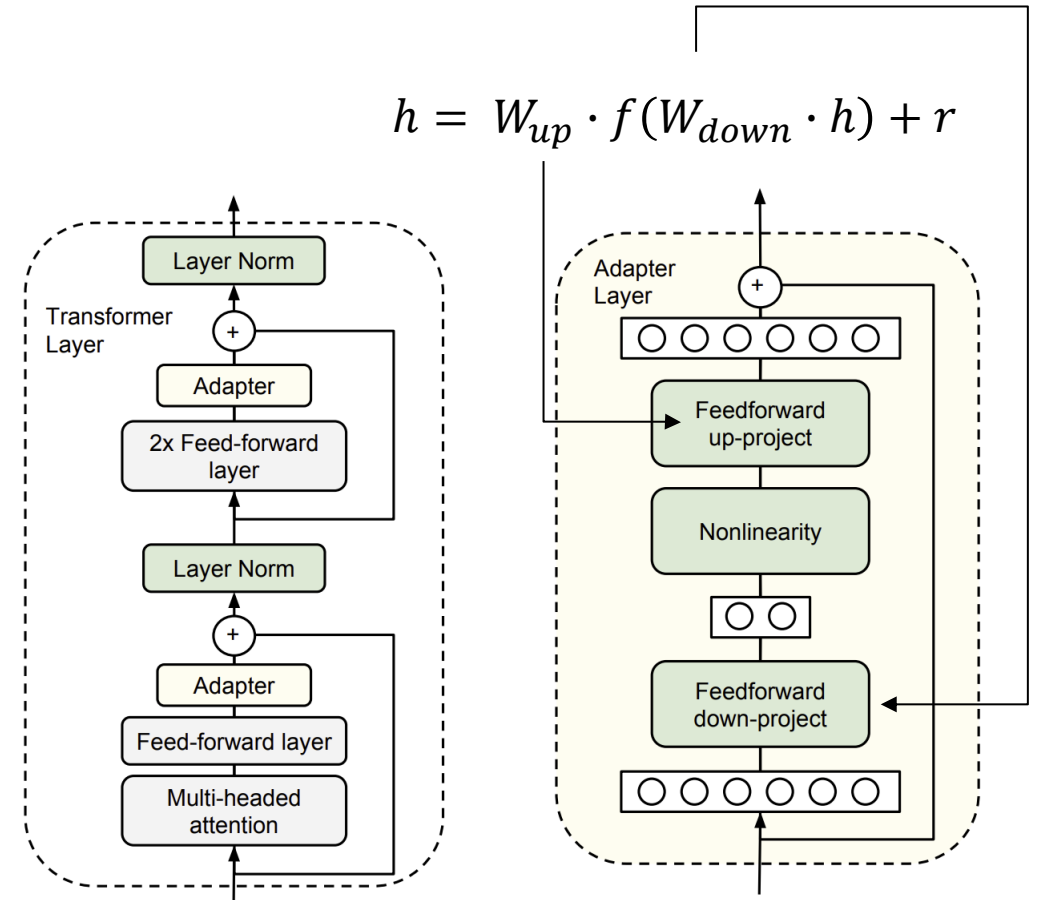
3. LLM FINE-TUNING METHODS

Parameter-efficient fine-tuning (PEFT)

- Adapt a model to a specific task or dataset and reduce the number of parameters to be trained
 - Lower computational & memory requirements, speed up adaptation process
- Various methods
 - Adapters (e.g. sequential bottleneck adapters and many more)
 - Prefix tuning
 - Low-rank adaptation (LoRA)
 - Quantized low-rank adaptation (QLoRA)

PEFT with Adapters

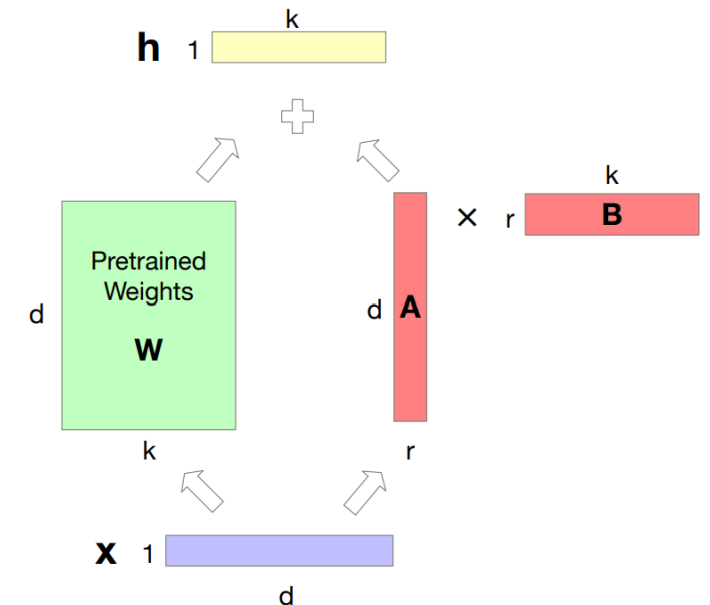
- Frozen base model is extended by a set of **trainable adapter components**
- Adapters consist of a bottleneck ($m \ll d$) and a skip-connection, adding 0.5 – 8% model parameters (i.e., $2 \cdot m \cdot d + d + m$)
- Training changes parameters of **adapter, layer normalization and final classification layer**
- Even with small m model latency may increase at inference without model parallelism



PEFT with Low-Rank Adaptation (LoRA)

- LoRA freezes the pretrained model weights and injects **trainable rank decomposition matrices** greatly reducing the number of trainable parameters with **minimal loss in model performance**
- LoRA can be used for **attention and FFNN matrices** in every layer of the Transformer-architecture (original papers uses it to decompose W_q, W_k, W_v)
- Approach: freeze W and **only fine-tune** the pair of **matrices A and B** of **low rank r** ($\ll \min(d, k)$)

$$h = x \cdot W' = x \cdot (W + A \cdot B)$$



d, k are input and output dimensions,
Number of parameters $r(d + k)$

PEFT with Low-Rank Adaptation (LoRA)

- Significant benefit from **reduction** in **storage** and **memory usage**
 - With $d = k = 4096$, W has 16'777'216 parameters; with $r = 4$, A and B have 16'384 parameters each
 - For GPT-3 175B and $r = 4$
 - **VRAM consumption** during **training** reduced from 1.2TB to 350GB (factor of 4)
 - **Checkpoint size** reduced from 350GB to 35MB (350GB still needed for model deployment; however, storing 100 adapted models only requires $350\text{GB} + 35\text{MB} * 100 \approx 354\text{GB}$ as opposed to $100 * 350\text{GB} \approx 35\text{TB}$)
- Additional benefits
 - **Models** customized to different tasks can be **swapped in and out on the fly** on machines that store the pre-trained weights in VRAM

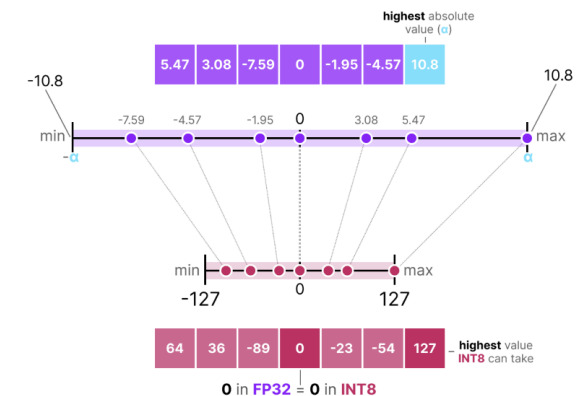
Quantized Low-Rank Adaptation (QLoRA)

- QLoRA saves memory without sacrificing performance with different steps, including
 - Training LoRA adapters in full precision
 - Freezing the base model and storing weights in a quantized form with 4-bit NormalFloat (symmetric post-training quantization), i.e., during backprop, gradients flow through them without changing their values
 - Quantizing scale values themselves used in quantization
- Using LoRA on all transformer layers (attention and FFNN) is critical to match 16-bit performance

Memory requirements $\sim \frac{N_{bits}}{8} \times N_{parameters}$

Full precision 32-bits = $\frac{32}{8} \times 70B \approx 280$ GB

Half precision 16-bits = $\frac{16}{8} \times 70B \approx 140$ GB



$$s = \frac{2^{b-1}-1}{\alpha}$$

$$s = \frac{127}{10.8} = 11.76$$

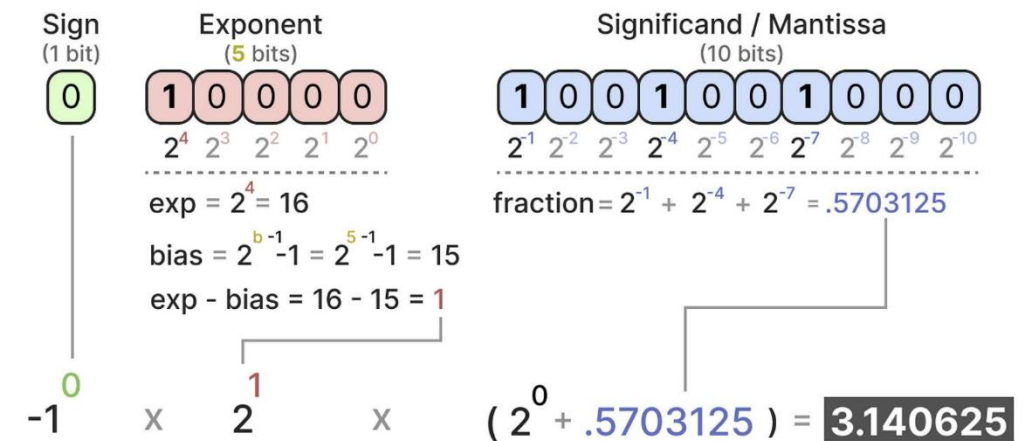
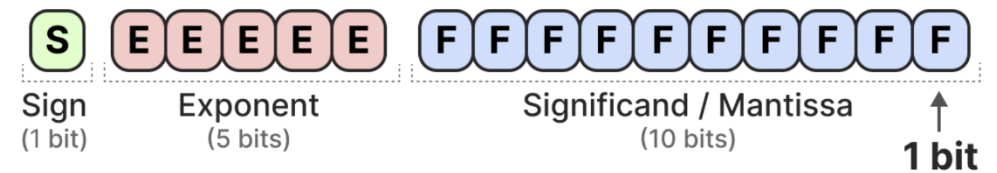
$$x_{\text{quantized}} = \text{round}(s \cdot x) \quad x_{\text{quantized}} = \text{round}(11.76 \cdot \text{value})$$

APPENDIX

LLM quantization basics (1)

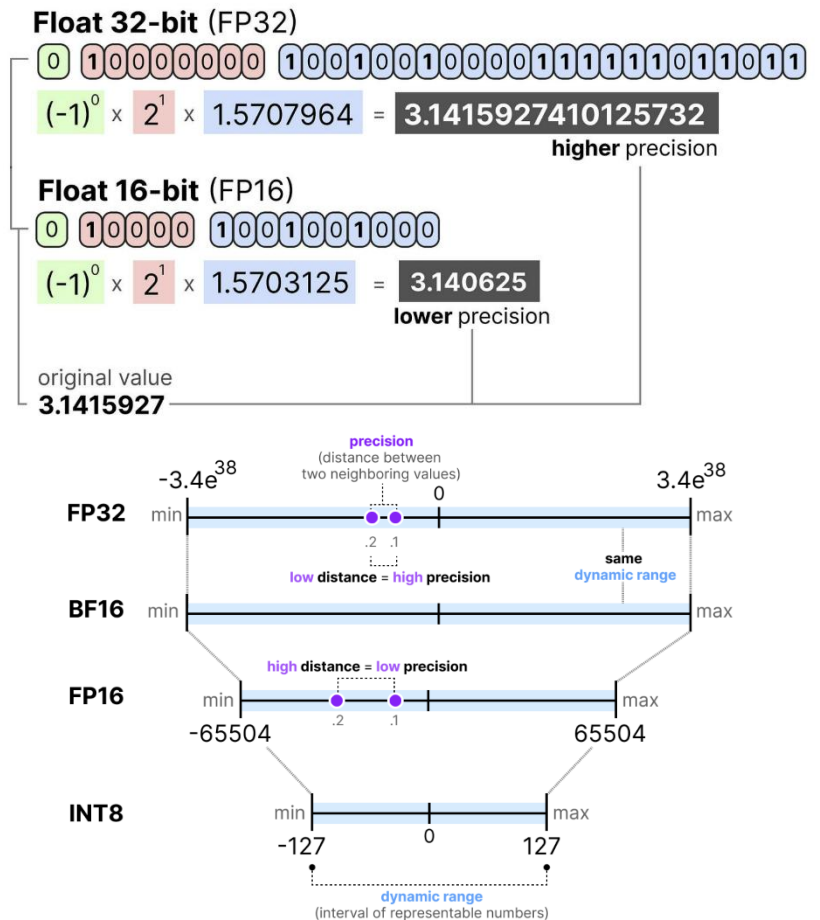
- LLMs typically have billions of parameters (mostly weights and less biases) which are expensive to store
- In addition, at inference a large number of activations are created as a product of the input and the weights
- All these value are initially represented as floating point numbers
- Quantization reduces **memory** requirements & **storage** cost, accelerates **computation** and lowers **power consumption**

Float 16-bit (FP16)



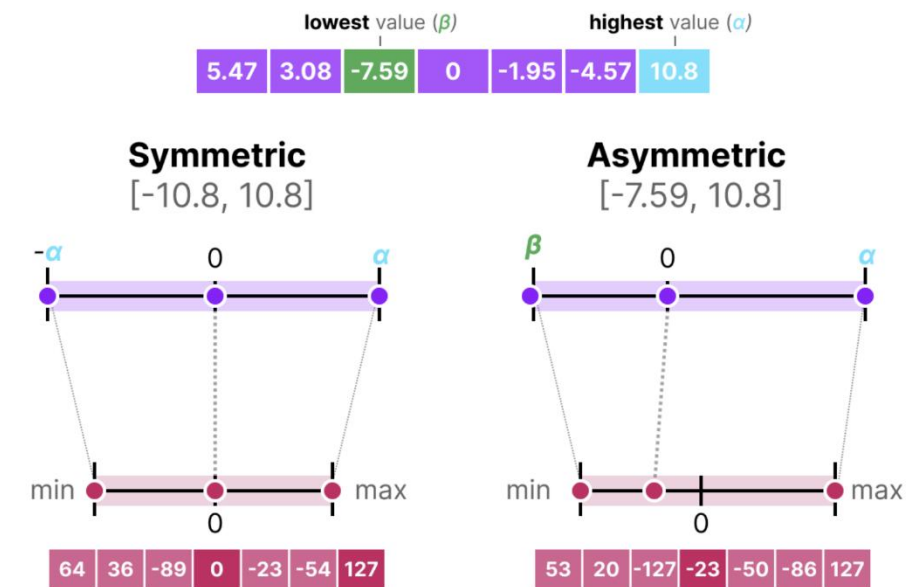
LLM quantization basics (2)

- Question: How can we reduce the number of bits representing these values while maintaining accuracy?
- Interval of numbers for a given representation called **dynamic range**, the distance between two neighboring values is called **precision**
- Rule of thumb: 4-bit (or higher) quantized models have a **good balance between compression and accuracy**



LLM quantization basics (3)

- **FFNN** static parameters, i.e., $O(10^9)$ **weights** and $O(10^6)$ **biases**
- **Intermediate activations** computed from weights and inputs which are sample-specific
- **Quantization approaches**
 - **Post-training quantization (PTQ) weight-only**
 - mapping weights to lower-precision values without additional data needed
 - **Post-training quantization (PTQ) weight and activations**
 - converting all values with additional calibration data **statically** (i.e., prepared once before inference) or **dynamically** (at inference at each hidden layer)
 - **Quantization-aware training (QAT)**



Quantization of weights performed with either symmetric or asymmetric quantization.
Quantization of activations, however, requires inference of the model to get their potential distribution since we do not know their range